pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

The Framework

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the framework

Days 2 to 8: your own code

`tool_schema`	the docstring, read
`agent_loop`	request, tools, repeat
`run_tools`	the message bookkeeping
`check_figures`	every figure, checked

Today

1. What a framework is for
2. Tracing off, and why
3. function_tool beside tool_schema
4. sdk_agent on the course's client
5. Day 2's question both ways: body_diff
6. await in a notebook
7. sdk_trace_table beside trace_table
8. What it does not do
9. LangGraph in one paragraph

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool, whoever builds the request

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book and the curve: numbers from your tools, never a view

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

What a framework is for

	Days 2 to 8	The framework
Tool schemas	tool_schema	function_tool
The loop	agent_loop	Runner.run
The stop rule	max_steps	max_turns
Bookkeeping	run_tools, tool_message	its own items
Checks on the answer	check_figures, yours	none

Story: the package's own record

openai-agents on PyPI: 0.0.1 uploaded 2025-03-04, yanked ("Updating to real release"); 0.0.2 on 2025-03-11.

The course pins 0.23.1, uploaded 2026-10-02; PyPI lists 123 versions with files.

Sources (read 2026-10-11): pypi.org/pypi/openai-agents/json; api.github.com/repos/openai/openai-agents-python

A framework writes four of the five parts. The fifth, the checks, stays yours.

Tracing off, and why

```
os.environ["OPENAI_AGENTS_DISABLE_TRACING"] = "1"   # before import agents: read once, at start
import agents
agents.set_tracing_disabled(True)                     # after: a manual setting wins from then on
```

NoOpTrace

what a new trace is now

two

switches, so import order cannot undo it

The default exporter, read in agents/tracing/processors.py:

`https://api.openai.com/v1/traces/ingest`

Nothing about a run leaves the machine unasked, and the saved runs hold only the agent's requests.

function_tool beside tool_schema

function_tool(dv01): the framework's schema


```
{"properties": {"settlement": {"description": "the settlement date as YYYY-MM-DD, for example 2026-10-02
.", "title": "Settlement", "type": "string"}, "maturity": {"description": "the maturity date as YYYY-MM-
DD.", "title": "Maturity", "type": "string"}, "coupon_pct": {"description": "the annual coupon in percen
t, 5.0 for a 5% coupon.", "title": "Coupon Pct", "type": "number"}, "yield_pct": {"description": "the yi
eld to maturity in percent, 5.06 for 5.06%.", "title": "Yield Pct", "type": "number"}}, "required": ["se
ttlement", "maturity", "coupon_pct", "yield_pct"], "title": "dv01_args", "type": "object", "additionalPr
```


tool_schema(dv01): day 1's


```
{"type": "object", "properties": {"settlement": {"type": "string", "description": "the settlement date a
s YYYY-MM-DD, for example 2026-10-02."}, "maturity": {"type": "string", "description": "the maturity dat
e as YYYY-MM-DD."}, "coupon_pct": {"type": "number", "description": "the annual coupon in percent, 5.0 f
or a 5% coupon."}, "yield_pct": {"type": "number", "description": "the yield to maturity in percent, 5.0
6 for 5.06%."}}, "required": ["settlement", "maturity", "coupon_pct", "yield_pct"], "additionalPropertie
```


The framework adds titles and nothing else. Equal once the titles are taken out: True.

sdk_agent on the course's client

```
desk_agent = agentkit.sdk_agent("desk", "Answer with the tool.", [dv01], aclient, agentkit.MODELS["desk"], 9)
```

```
instructions start with DESK_PROMPT: True; its own sentence: 'Answer with the tool.'
temperature 0.0, parallel_tool_calls False, store False, extra_args {'seed': 42, 'prompt_cache_key': 'cours...
tools ['dv01'], parameters sent ['settlement', 'maturity', 'coupon_pct', 'yield_pct']
```

How the framework chooses store, in its own source:

```
default_store = True if cls.is_openai(client) else None
return model_settings.store if model_settings.store is not None else default_store
```

On OpenAI's own address, store is true unless the settings say otherwise; elsewhere it is not sent.

Day 2's question, both ways: body_diff

body_diff(loop, framework)

field	in_a	in_b
messages	True	True
prompt_cache_key	False	True
store	False	True
tools	True	True

```
store: False; prompt_cache_key: 'course8-day09'
system message: DESK_PROMPT + '\nAnswer with the tool.'
the user message is the same: True
the tools are the same once the titles are taken out: True
temperature as sent: the loop 0, the framework 0.0
the two first requests have the same exchange key: no
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 951 prompt and 104 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 1,008 prompt and 126 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Added: prompt_cache_key and store. Temperature 0 against 0.0: equal as numbers, other bytes, another key.

Why sdk_settings sets two fields

```
default_agent = agents.Agent(..., model=agents.OpenAIChatCompletionsModel(model=..., openai_client=fake_defaults))
await agents.Runner.run(default_agent, QUESTION, max_turns=8)        # twice, each in its own cell
```

```
run 1: store True; prompt_cache_key begins agents-sdk:run: True
run 2: store True; prompt_cache_key begins agents-sdk:run: True
the two runs sent the same prompt_cache_key: no
the same question, the same exchange key: no
requests to a traces path: 0
```

store=False

the response is not kept

course8-day09

the same key on every run

A new random key per run changes every body, so every exchange key: a saved run is never found again.

await Runner.run in a notebook

```
outcome = await agents.Runner.run(desk_agent, QUESTION, max_turns=8)   # a notebook: one await per cell
asyncio.run(main())                                                    # a script of your own
Runner.run_sync(...)        # the scan refuses it in a notebook; its source says it cannot run here
```

```
asyncio.run: asyncio.run() cannot be called from a running event loop
requests sent by this cell: 0
```

coroutine

a run described, not started

one await

per cell: one failed request, one cell

A notebook already runs an event loop: await hands the run to it.

sdk_trace_table beside trace_table

```
the loop, trace_table:
 step role tool arguments finish_reason prompt_tokens completion_tokens
 0 user <NA> <NA>
 1 assistant dv01 {...} tool_calls 432 42
 1 tool dv01 <NA> <NA>
 2 assistant stop 519 62
the framework, sdk_trace_table:
 step role tool arguments finish_reason prompt_tokens completion_tokens
 0 user <NA> <NA>
 1 assistant dv01 {...} tool_calls 459 42
 1 tool dv01 <NA> <NA>
 2 assistant stop 549 84
the same columns: True
```

The same columns, read from the bodies the client sent: every check of the course works on both.

What the framework does not do: check a number

```
market_agent = agentkit.sdk_agent("market desk", ..., output_type=agentkit.DeskAnswer)
agentkit.check_figures(outcome_market.final_output, run_market)
```

```
Change in 10 Yr par yield between 2026-09-16 and 2026-10-02 27.0 bp 27.0 True True True True
2 Yr to 10 Yr slope on 2026-10-02 45.0 bp 45.0 True True True True
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 3,643 prompt and 262 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

```
loop       numbers no tool returned: []
framework  numbers no tool returned: []
```

The framework checks the JSON's shape. Whether a figure is its tool's value: your code, after the run.

LangGraph, Excel, and Git

LangGraph, as the course probed it on 2026-10-08

langgraph 1.2.14, langchain-core 1.6.7: MIT, released 2026-10-06, about 19.7 MB

Its nodes are your code: request bodies byte-identical to the raw loop's; replayed as shipped

LangSmith tracing off unless its variables are set. Not installed in the course.

Excel: no counterpart today. No agent framework, no tool requests; today's numbers were tested on days 1, 5, 6, 8.

```
agentkit.py | 144 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
test_agentkit.py | 87 ++++++++++++++++++++++++++++++++
2 files changed, 231 insertions(+)
```

git diff --stat HEAD~1 HEAD in the notebook; git diff --stat week1 HEAD in my_bondmath. 50 passed.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Tracing off, store false, the fixed key course8-day09.

2

Code back

Written for the course in the style of an AI answer. It answers.

3

Test first

No trace processor, a fake server, one run. It fails on store.

4

Fix

Tracing off; sdk_agent with sdk_settings inside. It passes.

Today: make_agent switches tracing on and leaves the defaults: store true, a random cache key every run.

Code that runs is not the same as code that is right.

What you can do now

- A framework writes schemas, the loop, the stop rule; not the checks
- Tracing off, twice, before and after import agents
- body_diff: the framework adds store and prompt_cache_key
- sdk_settings: store false, a fixed key, so runs replay
- One await per cell; check_figures on the framework's run

Week 2, Day 5 Case study: a macro agent on two statements